

PulseTrack AI

Uptime Monitoring + Status Page SaaS
with AI Incident Analyst

Complete Step-by-Step Roadmap • Every Endpoint • Every Feature • From Zero to Deployment
in Simple Words — 100% Free Stack — Intermediate Python Developer Level

Backend	Frontend	Infra	Cost
FastAPI + Postgres + Redis + Gemini	React + TS + Vite + Tailwind + Recharts	Docker + GitHub Actions + Render + Vercel	\$0 — Neon + Upstash + Resend + Supabase

**READ THIS
FIRST**

This is **not a todo app**. You build a real SaaS that pings websites every minute, creates incidents, sends alerts, shows a public status page, and uses Gemini AI to explain outages. Recruiters hire for this. Follow phases 0 to 10 in order. Do not skip testing, Docker and CI/CD — that is what makes you Intermediate.

Locked scope: 17 features • ~45 REST endpoints + 2 WebSockets • 8 weeks • Target: 80%+ test coverage | Generated: Sept 2026 |
Stack: Python 3.12, FastAPI, SQLAlchemy 2.0 Async, Alembic, uv, PostgreSQL 16, Redis 7, ARQ, React TS

• What Is Inside

- 1. Project in Simple Words — story of 1 outage
- 2. Final Locked Feature List — 17 features you will build
- 3. Tech Stack — what + why + free limit
- 4. How System Works — architecture + 3 flows in simple diagrams
- 5. Database Design — 10 tables, what each stores
- 6. Redis Design — 5 jobs Redis does
- 7. Complete API Contract — every endpoint with simple meaning
- 8. Workers & Scheduler — who pings, who alerts
- 9. Frontend Pages — 7 screens
- 10. Build Roadmap Phase 0–10 — exact tasks + done checklist
- 11. Testing + Docker + CI/CD + Free Deployment Map
- 12. Security, Caching, Rate-Limit Rules
- 13. Gemini AI — prompts + money-saving rules
- 14. 8-Week Timeline + Resume Lines + Interview Answers

HOW TO USE THIS DOC

Read sections 1–4 today. Then build **one phase at a time**. Each phase has a **Done =** box. Do not start next phase until Done is true. This is how real teams work.

01 Project in Simple Words

One-line idea

PulseTrack AI is a website where you add your website link, we watch it day and night, and if it goes down we tell you fast and AI tells you why. Your customers see a public green/red status page.

The 3 users

- **Owner / Admin** — creates team, adds members, sees billing limits, manages API keys and alerts.
- **Member / Agent** — adds monitors, acknowledges incidents, writes post-mortems.
- **Viewer / Customer (public)** — no login, only sees public status page */s/my-shop*.
- One team = one company. Swiggy data never mixes with Zomato data. This is called **multi-tenancy**.

Story of 1 outage — end to end

1. **You add monitor:** URL = **https://my-shop.com**, check every **1 min**, alert if keyword '**Checkout**' missing, track SSL.
2. **Worker pings every minute** and saves: **UP, 200, 180ms** in **checks** table. Dashboard dot stays green.
3. **2 AM site dies** — 2 fails in a row → we create **Incident #45 OPEN**. No human needed.
4. **Alert in 10 sec:** Email via Resend + Discord/Telegram webhook. If still down after 10 min → escalate to Admin.
5. **AI explains:** reads last 20 checks + error. Writes: '*SSL expired 2h ago. Fix: renew cert, restart Nginx.*' Cached 24h.
6. **You share status link** to customers. They see red + your update: '*We are fixing, back in 20 min.*'

7. **Site recovers** → 2 UP in a row → incident auto-resolves, downtime = 34 min, uptime % recalculated, weekly AI report updated.

SIMPLE RULE API never pings websites. **Worker pings**. API only shows data. Redis is the postman between them. If Gemini is slow, API still stays fast because AI runs in background.

02 Final Locked Feature List — 17 Features

We locked 8 core + 9 pro (added CSV export on request). Anything else goes to backlog. Do not add more during build.

Core 8 — must have

#	Feature	What it does (simple)
C1	Auth + Teams + Roles	Register/login with JWT. Teams isolate data. Roles: owner / member / viewer.
C2	Monitors CRUD	Add / edit / pause / delete URL monitors with interval 1/5/10 min.
C3	Periodic Checks	Worker pings URL, saves UP/DOWN + latency + code + error every interval.
C4	Incidents Auto	2 fails → OPEN incident. 2 success → RESOLVED + downtime calc.
C5	Alerts	Email + Discord/Slack/Telegram webhook with retry 3x. Test button.
C6	Dashboard + Charts	Green/red dots, latency p50/p95 graph, uptime % 30d/90d, incident timeline.
C7	Public Status Page	Shareable /s/slug, no login, cached 30s, incident history.
C8	API Docs + Health	Auto Swagger docs, /health, /metrics, pagination + filters everywhere.

Pro 9 — what makes you hired

#	Feature	What it does (simple) + why companies pay
P1	Heartbeat (reverse monitor)	Your cron pings us. If no ping in time → 'Cron dead' incident. For background jobs.
P2	Keyword + API + SSL checks	Check word exists, JSON field equals value, cert expiry days. Catches hacks.
P3	Escalation + Maintenance	Still down 10m → ping admin. Mute alerts during deploy window.
P4	Badge + Subscribe	Embed green badge in README. Visitors subscribe for UP emails.
P5	API Keys + Outbound Webhooks	Users automate via key. We POST signed JSON to their URL on incident.
P6	Deploy Markers	GitHub Action pings on deploy → vertical line on graph. Links outage to release.
P7	AI Analyst + Post-mortem	Explain outage + draft customer update + weekly report. Cached.
P8	Audit Logs + Limits	Who deleted what + when. Free plan = 10 monitors/team. Real SaaS logic.
P9	CSV Export (NEW)	One-click download: checks + incidents as CSV for Excel/Sheets. Boss-ready proof. \$0, just code.

BACKLOG — DO AFTER JOB Custom domains, SSO/OAuth Google, 2FA TOTP, multi-region checkers, SMS (Twilio paid), Stripe billing, mobile PWA push. Mention in README as *Future work* — shows vision without scope creep.

03 Tech Stack — Free Forever

Layer	Choice	Why + Free limit (simple)
Language	Python 3.12	Modern, async, type hints. Required for job.
API	FastAPI + Pydantic v2	Fast, auto Swagger docs, async. Best Python API framework now.
ORM + Migration	SQLAlchemy 2.0 Async + Alembic	Industry standard. Async = handles many checks at once.
Env	uv	Superfast venv + packages. Replaces pip + venv.
DB prod	Neon Postgres 16	Free 512 MB + pgvector for AI search. Enough for portfolio.
DB local	Docker Postgres 16	Same as prod, runs offline.
Cache/Queue	Upstash Redis (prod) Docker Redis 7 (local)	Free 10k cmds/day. Queue + cache + live updates + locks.
Worker	ARQ	Async Redis queue, made for FastAPI. Lighter than Celery.
Frontend	React + TS + Vite + Tailwind + shadcn/ui + TanStack Query + Recharts	Most in-demand. Decoupled SPA proves JWT + REST skills. Free on Vercel. Charts perfect for latency.
AI	Gemini 2.0 Flash Free	Free tier ~15 req/min. Only on-demand + cached. Never in hot loop.
Email	Resend Free	100 emails/day free. Enough for demo alerts.
Alerts extra	Discord / Slack webhook + Telegram Bot	100% free, unlimited. No SMS cost.
Files	None needed	No uploads in v1 — keeps it free + simple.
Infra	Docker + Compose GitHub Actions Render Free + Vercel Free	Render sleeps on free — mention cold start. Docker proves DevOps.

Folder structure you will use from Day 1 (this is what seniors expect):

```

backend/app/{main.py, core/{config.py, security.py, logging.py, rate_limit.py},
  api/v1/routers/{auth, teams, monitors, checks, incidents, ai, heartbeats,
    maintenance, notifications, api_keys, webhooks, status, deploys, audit}.py,
  models/, schemas/, services/, repositories/, workers/{scheduler, checker, alerter}.py}
frontend/src/{pages/{Login, Dashboard, MonitorDetail, Incidents, StatusPage},
  components/ui, hooks, lib/api-client.ts}
docker-compose.yml, Dockerfile, .github/workflows/ci.yml

```

04 How System Works — 3 Flows

A. Big picture

```

[React on Vercel] --REST+WS--> [FastAPI on Render] --SQL--> [Neon Postgres]
                                |--> [Upstash Redis: queue/cache/pubsub/lock]
                                [ARQ Worker] --ping--> [User Websites]
                                [ARQ Worker] --alert--> [Email/Discord/Telegram]
                                [FastAPI/Worker] --prompt--> [Gemini Free]

```

B. Monitoring loop (every minute)

1. Scheduler every 30s: find monitors where **next_check_at** <= **now** → push job **check(monitor_id)** to Redis.
2. Worker pops job, takes Redis lock **lock:monitor:{id}** (so 2 workers don't double-check).
3. Worker does **httpx GET timeout=10s**, measures latency, reads code + body + SSL days.

4. Decide UP/DOWN: DOWN if timeout / code >=500 / keyword missing / SSL <7 days (warning). Save to **checks**.
5. Set **next_check_at = now + interval**. Update cache **status:{id}**. Publish WS event.
6. Flap logic: look at last 2 checks. **DOWN+DOWN + no open incident** → **create incident + alert**. UP+UP + open incident → resolve.

C. Alert + AI flow (cheap)

- Alert sender reads **notification_channels** for team → sends Email + Webhooks in parallel → retry 3x with backoff → log to **alert_logs**.
- If still DOWN after 10 min and not acked → escalate to owner channel. If in **maintenance window** → skip alert, only log.
- AI only on button click: fetch last 20 checks + incident → check Redis **ai:{incident_id}** → if hit return in 50ms (\$0). Else 1 Gemini call → save to **incidents.ai_summary** + Redis 24h.

INTERVIEW LINE

"I keep pings out of request path. API returns fast. Workers scale separately. AI is cached so free quota never burns. Public status page is cached 30s so viral traffic can't kill DB."

05 Database Design — 10 Tables

Postgres stores truth. Redis stores speed. Never swap. All tables have **team_id** so teams can't see each other. Add index on **(team_id, ...)** everywhere.

Table	Key columns (simple meaning)	Relation
teams	id, name, slug (for status URL), plan (free/pro)	1 team → many users, monitors
users	id, team_id, email, password_hash, role, is_active	belongs to team
monitors	id, team_id, name, url, method, interval_min, keyword, json_path, ssl_check, is_paused, next_check_at	1 monitor → many checks, incidents
checks	id, monitor_id, status UP/DOWN, latency_ms, code, error, checked_at	append-only, partitioned by month, BRIN index
incidents	id, monitor_id, team_id, status OPEN/ACK/RESOLVED, started_at, ack_at, resolved_at, downtime_sec, ai_summary	1 incident → many comments, alerts
incident_comments	id, incident_id, user_id, body, created_at	timeline chat
heartbeats	id, team_id, name, ping_key, period_min, grace_min, last_ping_at, status	reverse monitor
maintenance_windows	id, team_id, monitor_id (null=all), starts_at, ends_at, reason	mute alerts in window
notification_channels	id, team_id, type email/discord/slack/telegram, config JSON, is_active	where to shout
api_keys + webhooks + audit_logs + deploys	api_keys: prefix, hash. outbound_webhooks: url, secret. audit: who did what. deploys: version, marked_at	platform + trust

Checks table will be biggest. 10 monitors × every 1 min = 14,400 rows/day. That's fine. Use **BRIN index on checked_at**, keep only 90 days (cron deletes old), and never JOIN full table — always filter by monitor_id + time range. Mention this in interview.

```

teams (1) --- (*) users                monitors (1) --- (*) checks (append-only)
teams (1) --- (*) monitors --- (*) incidents --- (*) incident_comments
teams (1) --- (*) heartbeats | maintenance | channels | api_keys | audit_logs

```

06 Redis Design — 5 Jobs

Use	Key example	Simple rule
1. Queue	arq:queue → check(12)	Scheduler pushes, worker pops. Retry 3x.
2. Cache	status:{monitor_id} TTL 60s public:{slug} TTL 30s, ai:{incident} TTL 24h	Read cache first. Delete on write (invalidate).
3. Lock	lock:monitor:{id} TTL 30s	Only 1 worker checks same monitor. Prevents double alerts.
4. Pub/Sub	channel:live → {monitor_id, status}	Worker publishes, API forwards to WebSocket. Live dot without refresh.
5. Rate-limit + blacklist	rl:{ip}:{min}, jwt:black:{jti}	AI 10/min, pings 60/min. Logout token blocked till expiry.

07 Complete API Contract — Every Endpoint

Base URL local: **http://localhost:8000/api/v1**. Auth: **Bearer JWT** except public status + heartbeat ping (uses key).
All lists support **?page&limit;&search;&status;&sort;**. All mutating routes write audit log.

HOW TO READ TABLE Method + Path → what it does in simple words. Build in this order. Starred (*) = demo wow.

Auth (5)

Method + Path	Auth	What it does (simple)
POST /auth/register {name,email,password,team_name}	No	Create team + owner user. Hash password bcrypt.
POST /auth/login {email,password}	No	Check password → return access 15m + refresh 7d.
POST /auth/refresh {refresh_token}	No	Rotate refresh, block reuse (reuse = theft).
POST /auth/logout	JWT	Blacklist access jti in Redis till expiry.
GET /auth/me	JWT	Return my profile + team + role.

Teams + Members (4)

Method + Path	Auth	What it does (simple)
GET /teams/me	JWT	My team info + plan + monitor count/limit.
PUT /teams/me {name}	Owner	Rename team / change status slug.
GET /teams/members	JWT	List members with roles.
POST /teams/invite {email,role}	Owner	Invite member (v1: direct add, v2: email link).

Monitors (7) — core

Method + Path	Auth	What it does (simple)
POST /monitors {name,url,interval,keyword}	Member+	Validate URL, enforce free limit 10, set next_check_at=now.
GET /monitors ?status	JWT	List with live cached status + last latency. Cached 60s.
GET /monitors/{id}	JWT	Detail + uptime % + last 20 checks. Check team owns it.
PUT /monitors/{id}	Member+	Edit URL/interval/keyword. Invalidate cache. Audit.
DELETE /monitors/{id}	Owner	Delete + cascade checks/incidents. Audit.
POST /monitors/{id}/pause	Member+	Skip scheduler until resume.
POST /monitors/{id}/resume	Member+	Set next_check_at=now.

Checks + Stats + Export (4)

Method + Path	Auth	What it does (simple)
GET /monitors/{id}/checks ?from&to;&limit;	JWT	Time-series rows for graph. Never full scan — paginated.
GET /monitors/{id}/stats ?days=30	JWT	Uptime %, avg/p95 latency, incidents count, MTTR. Computed from checks.
GET /monitors/{id}/checks/export ?from&to; (CSV)	JWT	Download checks as CSV: checked_at,status,latency_ms,code,error. Streaming, max 50k rows.
GET /incidents/export ?status (CSV)	JWT	Download incidents as CSV: id,monitor,started,resolved,downtime_min,ai_summary. For boss/Excel.

Incidents (7)

Method + Path	Auth	What it does (simple)
GET /incidents ?status	JWT	All incidents newest first, with monitor name.
GET /incidents/{id}	JWT	Detail + timeline + AI summary if exists.
POST /incidents/{id}/acknowledge	Member+	I am looking. Stops escalation timer.
POST /incidents/{id}/resolve {note}	Member+	Manual resolve + downtime calc.
GET /monitors/{id}/incidents	JWT	History per monitor for status page.
POST /incidents/{id}/comments {body}	Member+	Add timeline note. WS broadcast.
GET /incidents/{id}/timeline	JWT	Unified feed: created → alerts → acks → comments → resolved.

AI (4) *

Method + Path	Auth	What it does (simple)
POST /incidents/{id}/analyze	Member+	Gemini explains cause + fix. Cached 24h. Rate-limit 10/min.
GET /incidents/{id}/analysis	JWT	Return cached AI summary (50ms, \$0).
POST /incidents/{id}/draft-update {tone:friendly}	Member+	Draft customer status message to copy-paste.
POST /ai/ask {question}	Member+	Ask 'why down Tuesday?' — searches past incidents. 1 call.

Heartbeats (4)

Method + Path	Auth	What it does (simple)
POST /heartbeats {name,period}	Member+	Create reverse monitor, returns secret ping_key.
GET /heartbeats	JWT	List with status ALIVE/MISSING.
POST /heartbeats/{key}/ping	Key (no JWT)	Cron calls this. Updates last_ping_at. Public but unguessable.
DELETE /heartbeats/{id}	Member+	Delete heartbeat.

Maintenance + Channels + Keys + Webhooks (10)

Method + Path	Auth	What it does (simple)
POST /maintenance {monitor_id?,starts,ends}	Member+	Mute window. Scheduler checks before alert.
GET /maintenance / DELETE /maintenance/{id}	JWT	List / cancel window.
POST /notification-channels {type,config}	Member+	Add Email/Discord/Telegram. Encrypt secret.
GET /notification-channels	JWT	List channels + last delivery status.
POST /notification-channels/{id}/test	Member+	Send 'Test OK' to verify. Shows errors clearly.
POST /api-keys {name}	Owner	Create pk_live_... Return once. Store hash only.
GET /api-keys / DELETE /api-keys/{id}	Owner	List prefix + revoke.
POST /webhooks/outbound {url,secret}	Member+	We POST signed JSON on incident. HMAC header.
POST /webhooks/outbound/{id}/test	Member+	Send sample payload.
GET /audit-logs ?user&action;	Owner	Who did what, when. Immutable.

Public Status Page (4, no auth) *

Method + Path	Auth	What it does (simple)
GET /status/{slug}	Public	Team name + monitors live status. Cached 30s.
GET /status/{slug}/incidents	Public	Last 90d incidents for history bars.
POST /status/{slug}/subscribe {email}	Public RL	Subscribe for UP emails. Rate-limit strict.
GET /status/{slug}/badge.svg	Public	Green/red SVG for README. Cache 60s.

Deploys + System (4)

Method + Path	Auth	What it does (simple)
POST /monitors/{id}/deploys {version} + API-Key	API-Key	Mark deploy → vertical line on graph. From GitHub Action.
GET /monitors/{id}/deploys	JWT	List markers to overlay on chart.
GET /health	Public	200 {status:ok, db:up, redis:up}. Used by Render + CI.
WS /ws/monitors + /ws/incidents/{id}	JWT	Live push: status change, new comment. Via Redis pub/sub.

REQUEST / RESPONSE SHAPE (follow everywhere) Send JSON with Pydantic validation. Return `{data, meta:{page,total}, error:null}` or `{error:{code,message}}`. Errors: 400 bad input, 401 no login, 403 wrong role, 404 not found or not your team, 429 too many, 502 target site failed (not your bug).

08 Workers & Scheduler — Who Does What

- **Scheduler (every 30s, ARQ cron):** SELECT monitors WHERE next_check_at <= now AND not paused → enqueue **check_job**. Also enqueue **heartbeat_checker** (missing pings) + **ssl_checker** (daily) + **cleanup_old_checks** (daily, keep 90d).
- **check_job(monitor_id):** lock → httpx GET (10s timeout, follow redirects, UA PulseTrack) → measure ms → keyword/JSON/SSL asserts → save check → update next_check_at → cache+pub → incident logic → unlock. Timeout or exception = DOWN, never crash worker.
- **alert_job(incident_id):** load channels → send parallel (asyncio.gather) → Resend email + webhook POSTs → save alert_logs with success/error → schedule escalation check in 10m if still OPEN and not acked.
- **ai_job(incident_id, type):** build tiny prompt (last 20 checks only, ~800 tokens) → Gemini Flash → save + cache. If quota 429 → save 'AI busy, retry' and backoff, don't fail incident.
- **Scale story:** run 1 API + 2 workers locally via Compose. In interview: 'Workers scale horizontally because Redis lock prevents double work.'

ARQ jobs: check_job | heartbeat_job | alert_job | escalate_job | ai_job | cleanup_job
 Retries: check x0 (next tick will retry), alert x3 backoff 30s/2m/10m, webhook x3
 Timeouts: HTTP 10s, Gemini 20s, email 10s, total job < 45s

09 Frontend — 7 Screens + Export

Screen	What user sees + does	Key lib
Login / Register	Simple centered card, JWT stored httpOnly? v1: memory+localStorage, auto-refresh.	React Router, Zustand
Dashboard	Cards: Uptime now, Open incidents, p95 latency. Table of monitors with green/red dot auto-refresh 30s.	TanStack Query, Recharts
Monitor Detail	Big latency area chart + deploy markers + checks table + incidents list + Pause/Resume + Export CSV button.	Recharts, date-fns
Incidents	Filter OPEN/RESOLVED, click → timeline + Acknowledge/Resolve + AI Explain + Draft update + Export CSV.	Query + WS
Heartbeats + Maintenance	Cron ping URL with copy button + last ping ago. Calendar-ish mute list.	shadcn/ui
Settings: Channels, Keys, Webhooks	Add Discord URL → Test button shows green tick. Create API key → show once modal.	Forms + Zod
Public Status /s/slug *	No login. Big green/red banner, 90-day bars, subscribe input, badge copy snippet.	Vercel ISR, cached

CSV Export buttons: Monitor Detail → 'Export checks CSV' downloads checked_at,status,latency_ms,code,error. Incidents page → 'Export incidents CSV'. Uses StreamingResponse text/csv, filename pulsetrack-checks-{id}.csv. Design rule: dark slate header, white cards, green #16A34A for UP, red #DC2626 for DOWN, amber for ACK.

10 Build Roadmap Phase 0–10

**GOLDEN
RULE**

One phase = one PR merged to main. Each phase ends with tests passing + Docker up.
Never start next phase on broken main.

PHASE 0 — Setup (Day 1–2)

Init + Docker hello

- `uv init pulsetrack-ai`, `uv venv`, add `fastapi uvicorn sqlalchemy alembic pydantic-settings httpx arq redis pytest`.
- Init git, GitHub repo, `.gitignore`, `.env.example` (DATABASE_URL, REDIS_URL, JWT_SECRET, GEMINI_KEY).
- Write `docker-compose.yml`: `api` (hot reload), `db postgres:16`, `redis:7`. Verify `GET /health` returns ok.

Done =

Done = `docker compose up` works, `/health 200`, `/docs` opens.

PHASE 1 — Auth + Teams + RBAC (Week 1)

Login that is safe

- Models: `teams`, `users`. Alembic migration 001. Seed 1 team + owner.
- POST `register/login/refresh/logout` + GET `me`. Bcrypt hash, JWT 15m/7d, Redis blacklist.
- Dependency `require_role`. Write 8 pytest: login ok, wrong pass 401, viewer cannot delete (403), refresh reuse blocked.

Done =

Done = 8 auth tests green, Swagger login works, audit logs auth events.

PHASE 2 — Monitors + Checks Read (Week 2)

CRUD before magic

- Models `monitors` + `checks`. Migration 002 with indexes (`team_id`, `next_check_at`, BRIN `checked_at`).
- Build 7 monitor endpoints + 2 checks/stats (mock data first). Enforce 10/team limit + URL validator + audit.
- Frontend: dashboard table + add-monitor form. TanStack Query caching.

Done =

Done = create 10 monitors, pause/resume works, stats returns correct % on fake data.

PHASE 3 — Real Worker + Scheduler (Week 3) ★ HARDEST

The heart

- ARQ setup: scheduler cron 30s + `check_job` with `httpx 10s` timeout, UA header, redirect follow.
- Implement keyword + JSON-path + SSL-days asserts. Lock per monitor. Update `next_check_at`.
- Flap logic: 2x DOWN→OPEN, 2x UP→RESOLVE + `downtime_sec`. Test with `httpbin.org` + bad URL.

Done =

Done = add `https://example.com` 1min, see live checks rows growing, kill URL → incident OPEN in ~2 min.

PHASE 4 — Alerts + Escalation + Maintenance (Week 3–4)

Shout correctly

- Models `channels` + `alert_logs` + maintenance. Resend + generic webhook sender with `httpx` + retry.
- Alert on OPEN, resolve mail on RESOLVE. Escalate if OPEN 10m unacked. Skip if in maintenance.
- Channels UI + Test button. Subscribe to Discord webhook yourself to see real alert.

Done =

Done = downtime triggers real Discord msg + email, maintenance mutes correctly.

PHASE 5 — Live + Public Page + Heartbeat (Week 4)

Wow demo

- Redis pub/sub → WS /ws/monitors. React dot updates without refresh.
- Public /status/{slug} cached 30s + badge.svg + subscribe. Heartbeat create/ping/missing logic.
- Deploy markers POST + overlay on chart.

Done = Done = share status link in incognito (no login) works, badge green, cron ping demo works.

PHASE 6 — Gemini AI (Week 5) ★ DIFFERENTIATOR

Smart but cheap

- Prompt v1 analyze: 'You are SRE. Given 20 checks JSON, give: likely cause (1 line), evidence (2 bullets), fix (3 steps). Max 150 words.'
- draft-update prompt with tone param. ask endpoint with pgvector? v1: keyword search past incidents.
- Cache ai:{id} 24h + rate-limit 10/min. Handle 429 gracefully. UI: Explain button + copy draft.

Done = Done = click Explain → cached in 50ms second time, 1 Gemini call logged, quota safe.

PHASE 7 — Keys + Webhooks + Audit (Week 5–6)

Platform trust

- API keys: generate pk_live_ + hash sha256, middleware accepts Bearer pk_ OR JWT. Scope team.
- Outbound webhooks with HMAC X-Signature + test payload. Audit log on every write (user, action, target).
- Docs page with curl examples for keys + webhooks.

Done = Done = curl with API key creates monitor, webhook.site receives signed incident JSON.

PHASE 8 — Testing + Polish + CSV Export (Week 6)

Prove quality

- pytest 40+ tests: flap, escalation skip in maintenance, RBAC, cache hit, AI mocked, WS connect + CSV export content-type + team isolation. Coverage 80%+.
- Build CSV export: StreamingResponse text/csv with csv module, filter by team_id, limit 50k, filename header. Buttons in React download via blob URL.
- Ruff + Mypy + pre-commit. Seed demo data script (team Demo, 5 monitors, 1 open incident). Empty states, toasts, skeletons, mobile responsive.

Done = Done = pytest -q green, /checks/export returns valid CSV, coverage badge, demo login works for recruiter in 30s.

PHASE 9 — Docker + CI/CD (Week 7)

Ship like pro

- Multi-stage Dockerfile (slim, non-root, healthcheck). docker-compose.prod.yml.
- GitHub Actions: lint → mypy → pytest → docker build. Branch protection on main.
- Render Blueprint + Neon + Upstash env vars. Alembic migrate on deploy. Vercel env VITE_API_URL.

Done =

Done = git push → Actions green → Render auto-deploys → /health ok prod.

PHASE 10 — Launch + Portfolio (Week 7–8)

Get job

- README: live demo links (dashboard + status page), arch diagram, API table, test badge, \$0 cost table, demo creds.
- Loom 3-min video: add monitor → kill it → alert → AI explain → resolve. Add to LinkedIn + resume.
- Load test: 20 monitors 1min for 1h, screenshot p95 graph. Write post-mortem of your own outage.

Done =

Done = recruiter can click 2 links and understand in 60s without calling you.

11 Testing + Docker + CI/CD + Free Deploy

Testing — what to test

- **Unit:** decide_up_down(), stats calc, HMAC sign, prompt builder (no network).
- **API:** auth, RBAC deny, monitor limit 11th fails 403, public page no auth ok, private without token 401.
- **Worker:** mock httpx transport — DOWN+DOWN creates 1 incident (not 2), maintenance skips alert, resolve computes downtime.
- **Run:** `pytest -q --cov=app --cov-fail-under=80`. Mock Gemini + Resend, never hit real in tests.

Docker — simple

Dockerfile: python:3.12-slim → uv sync → non-root appuser → CMD uvicorn + alembic upgrade
Compose local: api:8000 + db:5432 + redis:6379 + worker (same image, CMD arq)
Compose prod: same image, env from Render/Neon/Upstash, no volumes

CI/CD pipeline

git push → GitHub Actions: ruff → mypy → pytest (postgres+redis services) → docker build test
main merge → deploy backend → Render (auto) → run migrations → deploy frontend → Vercel
PR must be green to merge. Add badges: build | coverage | python 3.12 | docker

Free deploy map — copy this

What	Where Free	Env var / note
Backend API + Worker (1 service)	Render Free (750h/mo)	1 instance only. No SSH/disk. See FREE CATCH below.
Postgres	Neon Free 512MB (use this, NOT Render Postgres)	DATABASE_URL pooled. Enable pgvector. Render Free DB expires in 30 days.
Redis	Upstash Free 10k/day (use this, NOT Render Key Value)	REDIS_URL. 10 monitors x 1min = ~3k cmds/day — safe. Render Free KV loses data on restart.
Frontend	Vercel Free	VITE_API_URL=https://your-api.onrender.com
Email	Resend Free 100/day	From onboarding@resend.dev for demo.
Alerts	Discord/Slack/Telegram	Unlimited free. Use for demo video.

**FREE CATCH
— RENDER
SLEEPS
AFTER 15 MIN
(LOCKED
DECISION:
STAY ON
FREE)**

Render Free web service **spins down after 15 min with no traffic**. You do **NOT restart manually** — it auto-wakes on next request in ~50 sec. **Catch for this project:** when asleep, your 1-min checks + worker also stop, so graphs will show gaps. **Free fix we will use:** ping **GET /health** every 14 min via **cron-job.org (free)** or **UptimeRobot (free)** to keep it awake. 1 always-awake service = ~720h/mo, fits in 750h free quota. Mention cold-start + keep-alive in README so recruiters know you understand free-tier trade-offs. Render Free Postgres/Key Value NOT used — Neon + Upstash stay always-on free.

```
Keep-alive (free): cron-job.org → GET https://your-api.onrender.com/health every 14 min
Render start CMD (free, 1 service runs both): web: alembic upgrade head && uvicorn app.main:app
+ arg worker in same service via start.sh (Render Free has no separate Background Worker)
Result: $0/mo, always-awake within 750h, checks keep running. Upgrade to $7/mo Starter only if you want no sleep.
```

12 Security, Cache, Rate-Limit — Rules to Memorize

Area	Rule (simple)
Auth	Bcrypt cost 12, JWT secret 32+ chars, access 15m, refresh rotate + reuse detection.
Tenancy	Every query filters team_id. Middleware loads team from JWT/API-key. Test cross-team 404.
Keys	Show key once, store SHA256 hash + prefix for lookup. Revoke = delete.
Webhooks	Sign body HMAC-SHA256 with secret, header X-Signature + timestamp, verify 5m skew.
Cache	Status 60s, public 30s, badge 60s, AI 24h. Invalidate on write. Never cache auth/me.
Rate-limit	Login 5/min/IP, AI 10/min/user, public subscribe 5/min/IP, ping 60/min/key. Return 429 + Retry-After.
Validation	Pydantic URL HttpUrl, interval in [1,5,10,30], keyword max 200 chars. SQLAlchemy bound params only.

13 Gemini AI — Prompts + Saving Money

Model: **gemini-2.0-flash**. Temperature 0.2 (factual). Max tokens 400. Timeout 20s. Always send **only last 20 checks**, never full history. Log tokens per call.

Prompt 1 — analyze (copy-paste ready)

```
You are a senior SRE. Given monitor {url} and 20 checks JSON [{at,code,latency,error}],
return: 1) Likely cause in 1 line 2) Evidence in 2 bullets 3) Fix in 3 steps.
Max 150 words. No guessing beyond data. If SSL days<14 mention it first.
```

Prompt 2 — draft-update

```
Write a {tone} 2-sentence customer status update for {site} down {mins} mins.
Cause: {ai_cause}. Include: we know, ETA, next update time. No jargon.
```

**SAVE QUOTA
— 3 RULES**

1) Cache 24h. 2) Never call in scheduler loop — only on click. 3) If 429, return cached or 'AI busy' — never crash incident. Weekly report = 1 call/week. This keeps you in free tier even with 50 demo clicks.

14 Timeline + Resume + Interview

8-week plan (10–12 hrs/week)

Week	Do	Show
1	Phase 0–1: setup + auth	Login works, tests 8 green
2	Phase 2: monitors CRUD	Dashboard lists monitors
3	Phase 3: worker live	Real checks saving every min
4	Phase 4–5: alerts + status	Discord alert video
5	Phase 6: AI	Explain button demo
6	Phase 7–8: keys + tests	Coverage 80% badge
7	Phase 9: Docker + CI/CD	Prod /health ok
8	Phase 10: polish + apply	README + Loom + 20 applications

Resume bullets — copy these after build

- Built multi-tenant uptime SaaS monitoring URLs every minute with **FastAPI async + ARQ workers + Redis queue**; p95 check <600ms, flap-safe alerting.
- Designed append-only **Postgres checks table (BRIN index, 90-day retention)** + Redis cache/pub-sub for live dashboard via WebSockets; public status pages cached 30s.
- Integrated **Gemini Flash RAG** for root-cause + post-mortems with 24h cache (90% fewer AI calls); shipped **Docker + GitHub Actions CI (80% pytest)** to Render/Vercel for \$0.

Interview answers — memorize

- **'How avoid double alerts with 2 workers?'** → Redis distributed lock per monitor + idempotent incident check (only if no OPEN exists).
- **'Checks table huge?'** → BRIN on time, filter by monitor+range, paginate, delete >90d via cron, never full JOIN.
- **'Gemini slow/429?'** → Async worker + 20s timeout + cache 24h + graceful fallback, API never blocks.
- **'Public page viral traffic?'** → Cached 30s in Redis, no auth DB hit, rate-limited subscribe, static badge.
- **'How do you know deploy broke site?'** → Deploy markers overlay on latency graph + AI correlates spike after marker.

NEXT STEP WHEN YOU SAY 'GO'

We start **Phase 0**: uv init + docker-compose + /health + first commit. I will give exact commands + files one by one. Reply **'start Phase 0'** when ready.

PulseTrack AI — Built to get you hired. Keep it simple, ship every week, show live links. Good luck.